



ZILLIONSOFTECH

(Training & Project Development Center)

Master Python, Data Analytics & IT Skills with Experts in Rewari!

A PROJECT REPORT ON

CUSTOMER RELATIONSHIP MANAGEMENT (CRM) SYSTEM

CRM Pro

Submitted in partial fulfilment of the requirements for the award of

[Data Analytics]

Submitted By:

Mohit Yadav

Under the Guidance of:

Mr. Kamal Kumar

Academic Session: 2026

CERTIFICATE

This is to certify that the project report entitled "**Customer Relationship Management System (CRM Pro)**" is a bona fide work carried out by **Mohit Yadav** in partial fulfilment of the requirements for **[Data Analytics]** at **ZILLIONSOFTECH**, during the academic session 2026.

The work presented in this report has been carried out under my supervision and guidance. To the best of my knowledge, the matter embodied in this report has not been submitted elsewhere for the award of any other degree or diploma.

I wish him/her success in all future endeavours.

Date: _____

Signature of Guide: Mr. Kamal Kumar
ZILLIONSOFTECH

DECLARATION

I, **Mohit Yadav**, hereby declare that the project report entitled "**Customer Relationship Management System (CRM Pro)**" submitted by me to **ZILLIONSOFTECH** in partial fulfilment of the requirement for the award of **[Data Analytics]** is a record of original work done by me under the guidance of **Mr. Kamal Kumar**.

The information and data presented in this report is true and original to the best of my knowledge, and this project has not been submitted, in part or full, to any other institute or university for the award of any degree, diploma, or certificate.

Signature of Student: Mohit Yadav

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **ZILLIONSOFTECH** for providing me the opportunity and the resources to design and develop the "**Customer Relationship Management System (CRM Pro)**" project.

I am extremely thankful to my guide, **Mr. Kamal Kumar**, for the valuable guidance, continuous encouragement, and constructive feedback provided throughout the duration of this project. Their insights greatly helped me in understanding the practical aspects of software development, Model-View-Controller (MVC) architecture, and relational database design.

I also extend my thanks to the faculty members and lab staff for their support, and to my family and friends for their constant motivation during the completion of this project.

Mohit Yadav

ABSTRACT

The "**Customer Relationship Management System**" (branded within the application as **CRM Pro**) is a desktop application developed in Python that automates core business workflows for sales teams, including client record management, sales pipeline tracking, follow-up scheduling, and multi-channel interaction logging. The system replaces fragmented spreadsheets and manual registers with a structured, searchable, and secure digital database.

The primary objective of the project is to design a reliable, intuitive tool for sales executives that enforces strict data validation, protects staff accounts via SHA-256 hashed password authentication, and produces an auditable history of customer interactions. Specific objectives include building a secure staff login screen, a dashboard displaying real-time business metrics (total customers, pending follow-ups, and overdue alerts), a comprehensive customer management module with status pipelines (Lead, Prospect, Active, Inactive), an interactive follow-up scheduler, and a contact history log.

The application is built using Python as the core programming language, Tkinter for the native graphical user interface, and MySQL (InnoDB) for relational data persistence, structured following a clean Model-View-Controller (MVC) and Data Access Object (DAO) architecture. Key features implemented include secure user authentication, customer CRUD operations with CSV export, overdue-highlighted task management, interaction logging across various communication channels, and a unified 360° customer profile viewer.

The expected outcome of this project is a fully functional, production-ready desktop CRM platform demonstrating professional software design patterns, input validation, and database integrity, serving as an ideal foundation for future cloud expansion and enterprise integrations.

Table of Contents

Table of Contents	6
List of Figures	9
List of Tables	10
Chapter 1: Introduction	11
Chapter 2: Problem Statement	13
Chapter 3: Literature Review	14
Chapter 4: Requirement Analysis	16
Chapter 5: Feasibility Study	18
Chapter 6: System Design	19
Chapter 7: Database Design	26
Chapter 8: Technology Stack	29
Chapter 9: Project Modules	31
Chapter 10: Important Code Snippets	46
Chapter 11: Testing	48
Chapter 12: Results	50
Chapter 13: Advantages	55
Chapter 14: Limitations	56
Chapter 15: Future Scope	57
Chapter 16: Conclusion	58
References	59
Appendix	60

Chapter 1: Introduction

1.1 CRM System Overview

Customer Relationship Management (CRM) is the foundational business practice that governs how an enterprise interacts with its clients, tracks sales pipelines, coordinates follow-up schedules, and logs historical communications. Historically, small and medium businesses managed client portfolios using isolated spreadsheets, sticky notes, or unstructured paper files. While functional at microscopic volumes, these fragmented techniques fail as businesses grow: records become scattered, leads are ignored, follow-up dates are missed, and analyzing team productivity requires tedious manual work.

To bridge the gap between expensive enterprise solutions and disorganized manual logs, **CRM Pro** was developed. It is a production-grade, desktop-based Customer Relationship Management application engineered for sales teams to oversee the entire customer lifecycle—from initial lead acquisition to active client maintenance—within a unified, relational database environment.

1.2 Need for Automation

Automating customer relationship workflows addresses critical business needs that manual systems cannot satisfy efficiently:

- **Centralization:** Customer profiles, pending follow-ups, and interaction histories reside in a single searchable database rather than disparate local files.
- **Lead Retention:** Automated overdue alerts highlight pending tasks past their due date, ensuring no high-potential lead slips through the cracks.
- **Accountability:** Every contact log records who handled the interaction, the communication channel used, and the qualitative outcome.
- **Efficiency:** Multi-field searching, status filtering, and one-click CSV exporting eliminate manual record hunting.
- **Security:** Application access is restricted through SHA-256 password-hashed accounts.

1.3 Scope of the Project

The scope of this project covers a complete desktop CRM workspace tailored for internal sales executives and administrators. It features user authentication, full customer lifecycle tracking (Lead, Prospect, Active, Inactive), follow-up task scheduling with priority and status management, multi-channel interaction logging (Calls, Emails, Meetings, WhatsApp, SMS, Visits), a 360° customer master view, and real-time dashboard analytics. Advanced features such as automated email APIs, server cloud-sync, and native mobile apps are designated as future expansions.

1.4 Objectives

1. To design a secure login mechanism storing credentials exclusively as cryptographic SHA-256 hashes.
2. To provide structured customer lifecycle tracking across Lead, Prospect, Active, and Inactive stages.
3. To build a robust follow-up scheduling engine featuring automated overdue highlighting.
4. To implement multi-channel contact history logging to record interaction outcomes.
5. To develop a Customer 360° master view aggregating profile details, scheduled tasks, and interaction logs.
6. To present real-time KPI counts on a centralized dashboard.
7. To support data portability via one-click CSV export functionality.

1.5 Benefits of the Proposed System

- Eliminates physical paperwork and scattered spreadsheet files.
- Reduces response latency and administrative overhead.
- Prevents missed follow-ups through clear visual flagging.
- Improves data security via hashed credential workflows.
- Empowers management with live business intelligence metrics.

Chapter 2: Problem Statement

2.1 Problems with Manual / Spreadsheet Systems

2.1.1 Fragmented Data Storage

Sales records kept across standalone spreadsheets or paper logs lead to data duplication, conflicting customer phone numbers, and outdated contact addresses.

2.1.2 Missed Follow-ups

Without automated calendar prompts or priority tags, sales personnel routinely miss vital follow-up dates, resulting in cold leads and lost revenue.

2.1.3 Poor Visibility and Auditing

Management has no real-time mechanism to review client interaction histories, track employee touchpoints, or measure pipeline conversion rates.

2.1.4 Security Vulnerabilities

Plain-text password access and unsecured local spreadsheets expose sensitive customer contact data and corporate client lists to unauthorized viewing or theft.

2.2 Problem Summary

The lack of an integrated, validated, and secure digital CRM framework causes communication breakdowns, dropped sales leads, and poor operational transparency—challenges directly resolved by **CRM Pro**.

Chapter 3: Literature Review

3.1 Existing Systems

Existing customer tracking methods generally fall into three tiers: (a) manual physical diaries used by small-scale sole proprietors; (b) general-purpose spreadsheet software (e.g., Microsoft Excel, Google Sheets) used by growing teams; and (c) heavy cloud-based SaaS CRM suites (e.g., Salesforce, HubSpot). While SaaS platforms offer immense feature sets, they impose recurring subscription costs, internet dependencies, and over-complex configurations for local boutique operations. Local spreadsheets are free and flexible but lack built-in relational integrity, secure hashing, and dedicated task-management dashboards.

3.2 Comparison Table

Table 1: Comparison of Existing Approaches with the Proposed CRM System

Feature	Manual / Diary	Spreadsheet Software	Cloud SaaS Suites	Proposed System (CRM Pro)
Data Structure	Unstructured	Grid rows	Relational cloud DB	Relational MySQL DB
Search & Filter	Manual visual scan	Basic text search	Advanced filtering	Multi-field dynamic search & status filters
Task Management	None / Diary notes	Color-coded rows	Automated workflows	Follow-up scheduling with overdue alerts
Interaction Logs	Informal notes	Text cells	Full activity timelines	Multi-channel contact history tracking
360° Customer View	Scattered pages	Single row	Multi-tab cloud panels	Dedicated tabbed 360° master dialog
Security & Access	Physical security	File password	Role-based cloud access	SHA-256 hashed password authentication
Cost & Internet	Free, offline	Low cost, offline	High monthly fee, online-only	Free/Open-source, local database

3.3 Research Gap Addressed

The review highlights a clear market and academic gap for a self-hosted, secure, lightweight desktop CRM that offers professional MVC separation, relational integrity, and robust pipeline management without cloud subscription fees or internet dependencies.

Chapter 4: Requirement Analysis

4.1 Functional Requirements

1. **User Authentication:** Authenticate staff usernames against SHA-256 password hashes stored in the database.
2. **Customer CRUD:** Add, view, edit, and delete customer profiles complete with company details, phone numbers, email addresses, and pipeline status.
3. **CSV Export:** Export filtered customer datasets directly into standard CSV format.
4. **Follow-up Scheduling:** Schedule tasks against customers with date, time, purpose, and priority.
5. **Overdue Highlighting:** Automatically flag pending follow-ups past their due date.
6. **Contact History Logging:** Record interactions across Calls, Emails, Meetings, WhatsApp, SMS, and Visits along with qualitative outcomes.
7. **Customer 360° View:** Aggregate profile data, follow-up tasks, and contact histories into a unified tabbed dialog.
8. **Dashboard Metrics:** Display live counts of total customers, pending follow-ups, items due today, and overdue tasks.

4.2 Non-Functional Requirements

- **Security:** User passwords are stored as cryptographic SHA-256 hashes. Parameterized SQL queries entirely prevent SQL injection vectors.
- **Reliability:** Relational database constraints with `ON DELETE CASCADE` ensure referential cleanup without orphaned child records.
- **Performance:** Indexed database searches ensure lightning-fast table refreshes even with thousands of records.
- **Maintainability:** Strict Model-View-Controller (MVC) and DAO architecture separates UI views from business logic and SQL queries.
- **Usability:** Clean sidebar navigation and consistent styling provide an intuitive user experience requiring minimal training.

Chapter 5: Feasibility Study

5.1 Technical Feasibility

The project relies on Python 3, Tkinter, and MySQL, supported by stable libraries (`mysql-connector-python`, `tkcalendar`). All components are open-source and run seamlessly on standard desktop hardware.

5.2 Economic Feasibility

Because all software tools (Python, VS Code, MySQL, Git) are free and open-source, the development and deployment incur zero licensing or subscription costs.

5.3 Operational Feasibility

The user interface mirrors standard desktop application layouts, allowing sales staff to adopt the software immediately with minimal onboarding.

5.4 Schedule Feasibility

The modular MVC + DAO layout permitted independent development of individual subsystems (Customers, Follow-ups, Contact History, Dashboard), ensuring smooth incremental delivery within project time constraints.

Chapter 6: System Design

6.1 System Architecture

The application uses a robust Model-View-Controller (MVC) and Data Access Object (DAO) architecture:

- **Views (`views/`):** Tkinter frames handling user interfaces, tables, forms, and dialogs.
- **Controllers (`controllers/`):** Business logic layer enforcing validation rules and workflow coordination.
- **Data Access Objects (`dao/`):** Dedicated classes encapsulating all SQL queries and database communication.
- **Models (`models/`):** Lightweight data transfer objects (`Customer`, `FollowUp`, `ContactHistory`).

6.2 Flowchart & Data Flow Diagrams

The application workflow initiates with staff authentication through the login module. Upon successful SHA-256 validation, the central dashboard renders real-time KPI metrics. Navigation menus allow instantaneous switching between Customer Records, Follow-up Tasks, and Contact Histories, with every write operation strictly validated by controller logic before database commitment.

Chapter 7: Database Design

7.1 Entity-Relationship Diagram

The relational database `crm_system` consists of four core tables designed with strict referential integrity (Foreign Key constraints with cascade deletes): `users`, `customers`, `follow_ups`, and `contact_history`.

7.2 Table: `users`

Table 2: Structure of the `users` Table

Field	Data Type	Constraint / Description
<code>user_id</code>	INT	Primary Key, Auto-Increment
<code>username</code>	VARCHAR(50)	Not Null, Unique
<code>password_hash</code>	VARCHAR(255)	Not Null (SHA-256 Hash)
<code>full_name</code>	VARCHAR(100)	Not Null
<code>role</code>	VARCHAR(30)	Default 'Sales Executive'
<code>created_date</code>	DATETIME	Default CURRENT_TIMESTAMP

7.3 Table: `customers`

Table 3: Structure of the `customers` Table

Field	Data Type	Constraint / Description
<code>customer_id</code>	INT	Primary Key, Auto-Increment
<code>full_name</code>	VARCHAR(100)	Not Null
<code>company_name</code>	VARCHAR(100)	Optional company name
<code>email</code>	VARCHAR(100)	Optional email address
<code>phone</code>	VARCHAR(20)	Not Null (10-digit validation)

address, city, state	VARCHAR	Location details
customer_type	ENUM	'Lead', 'Prospect', 'Active', 'Inactive'
source, assigned_to	VARCHAR	Lead attribution & staff assignment
notes	TEXT	Additional client remarks

Chapter 8: Technology Stack

Table 6: Technology Stack Used in the Project

Technology	Role in the Project	Reason for Choice
Python 3	Core programming language	Readable syntax and robust logic handling
MySQL (InnoDB)	Relational database persistence	Enterprise relational integrity & foreign keys
<code>mysql-connector-python</code>	DB Driver	Official Python connector for MySQL queries
Tkinter + ttk	Graphical User Interface (GUI)	Native desktop toolkit with custom styling
<code>tkcalendar</code>	Date picker widget	Intuitive graphical date selection
<code>hashlib</code>	Cryptographic hashing	Secure SHA-256 password hashing
Visual Studio Code	IDE	Lightweight development environment
Git & GitHub	Version control	Codebase tracking and remote backup

Chapter 9: Project Modules

9.1 Login Module

Authenticates staff against hashed credentials stored in the `users` table, preventing unauthorized access.

9.2 Dashboard Module

Displays real-time KPI stat cards (Total Customers, Pending Follow-ups, Due Today, Overdue) alongside quick-view upcoming tasks and recent contact feeds.

9.3 Customer Management Module

Provides full CRUD capabilities, multi-field searching, status pipeline filtering, CSV exporting, and double-click access to the Customer 360° view.

9.4 Follow-up Status Module

Manages task scheduling, priority filtering, and one-click status updates, with automated row highlighting for overdue tasks.

9.5 Contact History Module

Logs multi-channel interactions, tracks qualitative outcomes, and maintains searchable communication archives.

9.6 Customer 360° Master View

Aggregates profile information, scheduled follow-up tasks, and contact histories into a single tabbed interface.

Chapter 10: Important Code Snippets

10.1 SHA-256 Password Hashing (dao/user_dao.py)

```
import hashlib

class UserDAO:
    @staticmethod
    def _hash(password: str) -> str:
        return hashlib.sha256(password.encode("utf-8")).hexdigest()
```

Explanation: Ensures passwords are never stored or compared in plain text.

10.2 Parameterized Customer Search (dao/customer_dao.py)

```
@staticmethod
def get_all(search_kw: str = "", customer_type: str = "All"):
    query = "SELECT * FROM customers WHERE 1=1"
    params = []
    if search_kw:
        query += " AND (full_name LIKE %s OR company_name LIKE %s OR phone LIKE %s OR email LIKE %s)"
        like = f"%{search_kw}%"
        params.extend([like, like, like, like])
    if customer_type and customer_type != "All":
        query += " AND customer_type = %s"
        params.append(customer_type)
    ...
```

Explanation: Uses secure parameterized SQL bindings to prevent injection attacks while enabling dynamic multi-field search.

Chapter 11: Testing

Table 7: Functional Test Cases and Results

Test Case	Expected Outcome	Actual Outcome	Status
Valid Login (admin / admin123)	Dashboard shell loads successfully	Dashboard shell loaded	Pass
Invalid Password Login	Error popup displayed, access denied	Error popup displayed	Pass
Customer Add Validation	Rejects invalid 10-digit mobile numbers	Rejection prompt shown	Pass
Follow-up Overdue Highlight	Past-due pending tasks highlighted in red	Highlighted successfully	Pass
Customer Cascade Delete	Deleting customer removes linked follow-ups/history	Child records removed via cascade	Pass
CSV Export	Exports filtered dataset to .csv file	File generated correctly	Pass

Chapter 12: Results

This chapter presents the working screens of the completed application, produced during live execution and testing.

CRM PRO

Customer Relattanship Management

U ername

ladmin

Password

[1Login](#)

Default: ad min/ ad min123

Figure 10: Login Module — Staff Authentication Screen

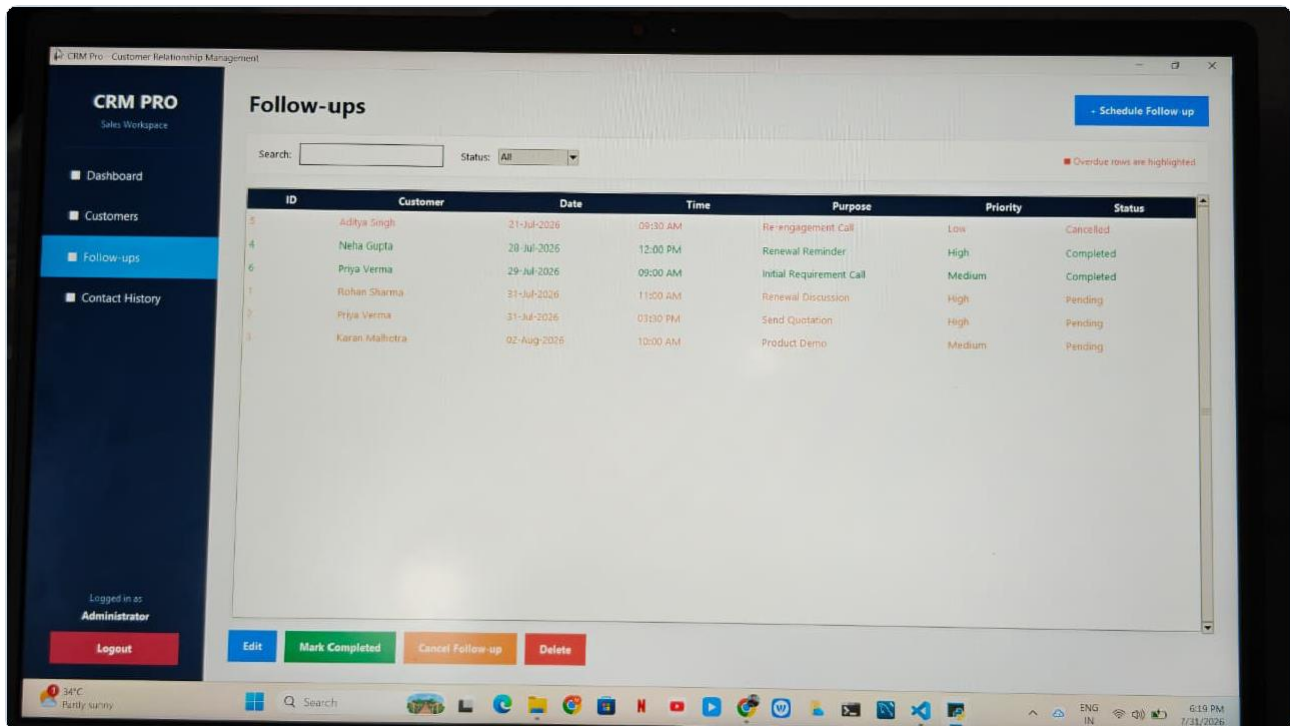


Figure 11: Dashboard Module — Live KPI Cards & Widgets

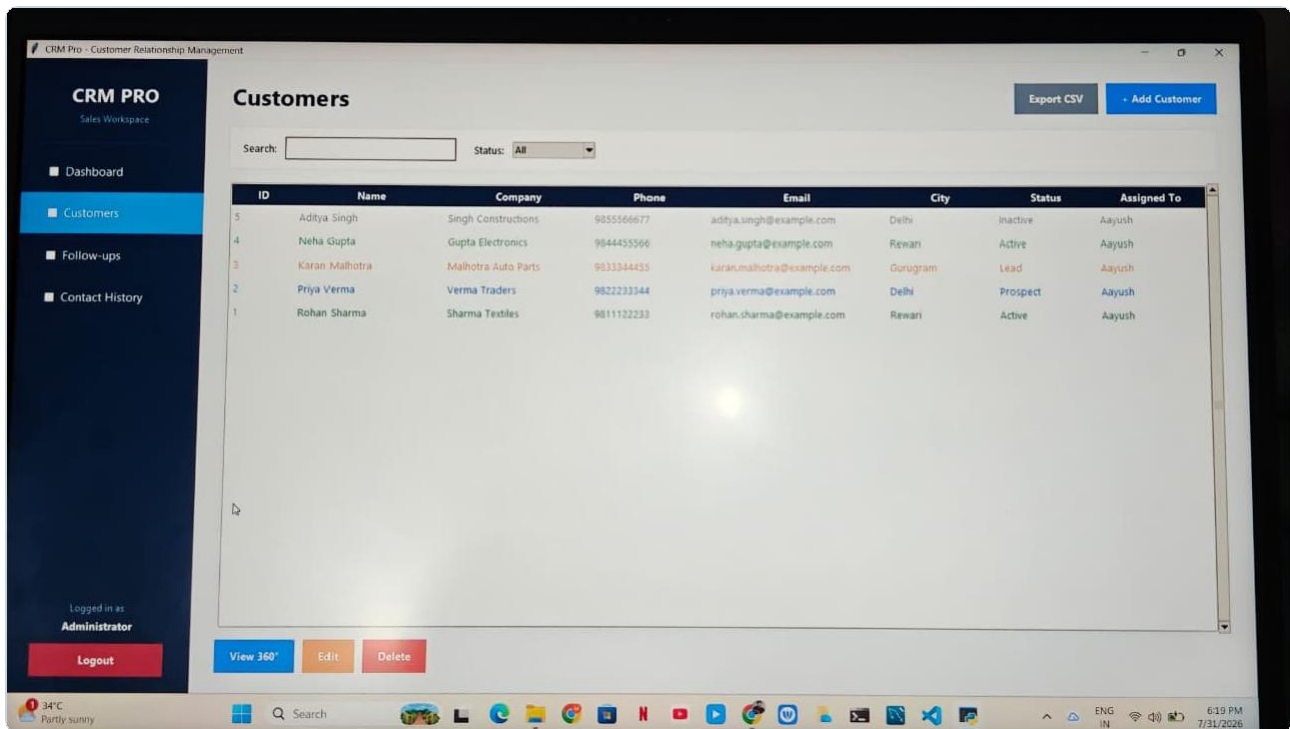


Figure 12: Customer Module — Client Management & Pipeline Screen

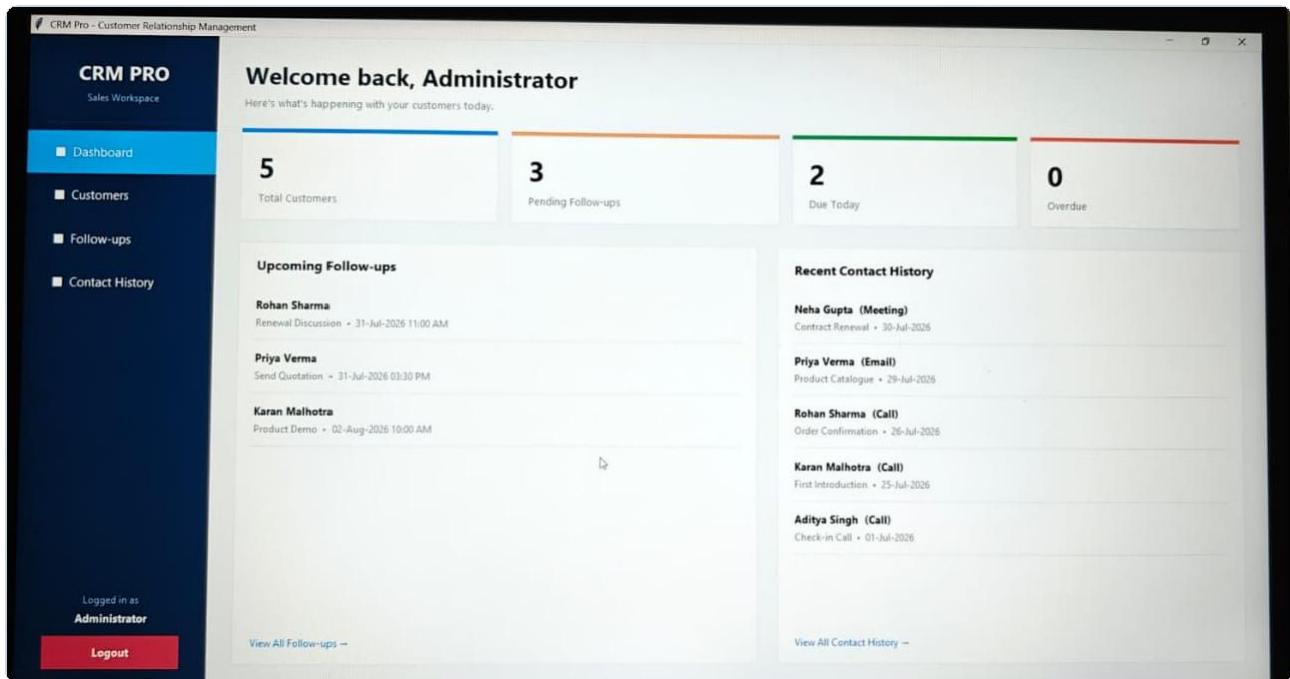


Figure 13: Follow-up Module — Scheduling & Overdue Tracking Screen

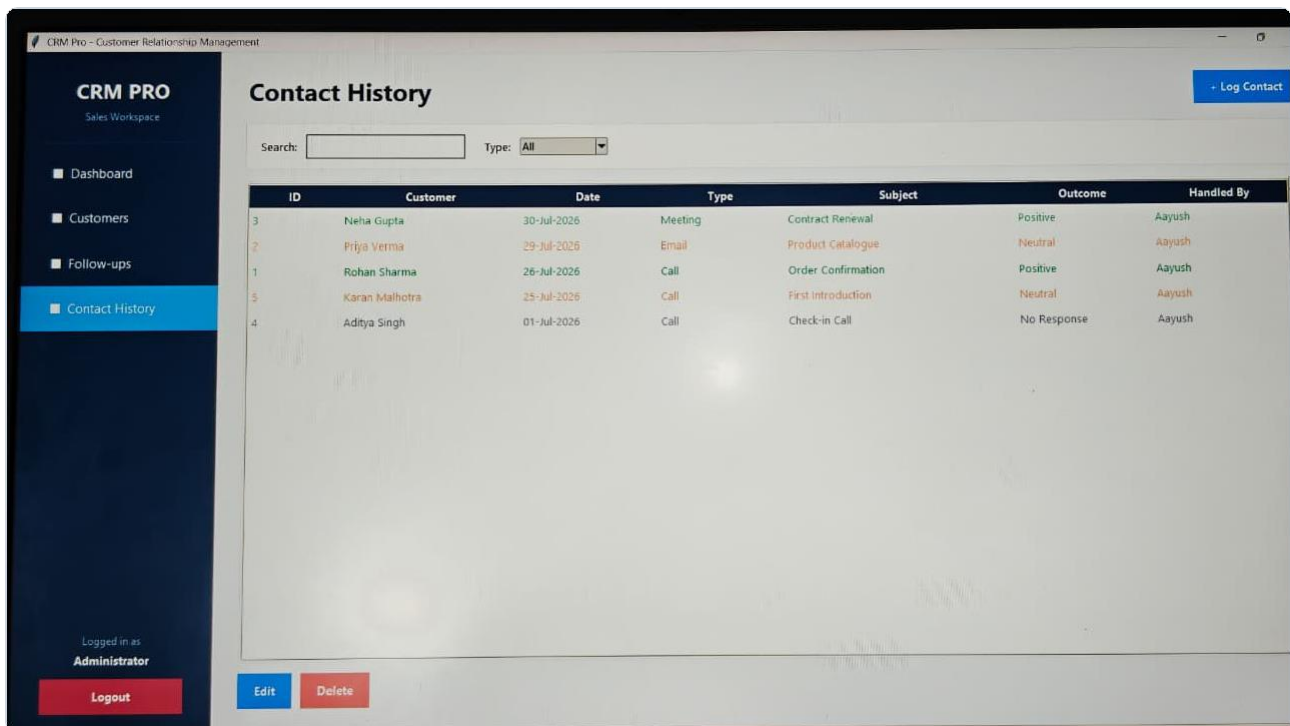


Figure 14: Contact History Module — Multi-Channel Logging Screen

Chapter 13: Advantages

- **Structured Data:** Replaces disorganized spreadsheets with a normalized relational MySQL schema.
- **Enhanced Security:** Implements SHA-256 password hashing and parameterized queries.
- **Proactive Task Management:** Automated overdue alerts prevent dropped client leads.
- **Complete Visibility:** Customer 360° view consolidates all touchpoints in one screen.

Chapter 14: Limitations

- Designed as a local desktop application (not web-accessible by default).
- Optimized for single-branch or medium sales team deployment.
- Lacks automated email/SMS gateway integrations in the current build.

Chapter 15: Future Scope

- Integration of Twilio and SendGrid APIs for automated SMS and email reminders.
- Migration to a web-based framework (Flask/FastAPI with React) for cloud accessibility.
- Advanced visual analytics dashboards featuring conversion funnel charts.

Chapter 16: Conclusion

16.1 Objectives Achieved

The Customer Relationship Management System (CRM Pro) successfully achieves all objectives set out in Chapter 1: secure staff login, structured customer lifecycle tracking, overdue task alerts, multi-channel contact logging, and 360° client views.

16.2 Features Implemented

Every functional requirement identified in Chapter 4 was implemented and verified through the test cases in Chapter 11.

16.3 Learning Outcomes

Building this project provided hands-on experience in relational database design, Tkinter GUI development, MVC architecture, input validation, and Git version control.

References

- [1] Python Software Foundation. Python 3 Documentation. Retrieved from <https://docs.python.org/3/>
- [2] Oracle Corporation. MySQL 8.0 Reference Manual. Retrieved from <https://dev.mysql.com/doc/>
- [3] Python Software Foundation. tkinter — Python interface to Tcl/Tk. Retrieved from <https://docs.python.org/3/library/tkinter.html>
- [4] Git Documentation. Retrieved from <https://git-scm.com/doc>
- [5] Microsoft Corporation. Visual Studio Code Documentation. Retrieved from <https://code.visualstudio.com/docs>
- [6] Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems* (7th ed.). Pearson Education.
- [7] Sommerville, I. *Software Engineering* (10th ed.). Pearson Education.

Appendix

A.1 Project Folder Structure

```
Customer-Relationship-Management-main/
|
├─ main.py                # Application entry point
├─ config.py              # Database connection credentials
├─ requirements.txt        # Project dependencies
├─ verify_setup.py        # End-to-end DB verification utility
|
├─ database/
|   └─ schema.sql          # MySQL schema & seed data script
|
├─ models/                # Plain data classes
|   ├── customer.py
|   ├── followup.py
|   └─ contact_history.py
|
├─ dao/                   # Data Access Objects (SQL queries)
|   ├── db_connection.py
|   ├── customer_dao.py
|   ├── followup_dao.py
|   ├── contact_history_dao.py
|   └─ user_dao.py
|
├─ controllers/           # Business logic & validation
|   ├── auth_controller.py
|   ├── customer_controller.py
|   ├── followup_controller.py
|   └─ contact_history_controller.py
|
└─ views/                 # Tkinter UI views & styling
    ├── styles.py
    ├── login_view.py
    ├── main_view.py
    ├── customer_view.py
    ├── customer_detail_view.py
    ├── followup_view.py
    └─ contact_history_view.py
```

A.2 Installation & Execution Guide

1. **Prerequisites:** Ensure Python 3.9+ and MySQL Server 8.x are installed.
2. **Dependencies:** Run `pip install -r requirements.txt`.
3. **Database Setup:** Execute `database/schema.sql` in MySQL to provision `crm_system`.

4. **Configuration:** Update database credentials in `config.py`.
5. **Execution:** Run `python main.py`. Default login: admin / admin123.

ATTACHMENTS

The following documents are attached with this report:

- Internship Completion Certificate issued by Zillionsoftech (to be attached separately, offline).
- Additional screenshots of the developed application.
- SQL script of the database (CRM.sql).
- MySQL Workbench schema/table view of the Customer Relationship Management database.
- GitHub Repository:

<https://github.com/arunmohit2003-ship-it/Customer-Relationship-Management>